IoT Network / Memory Stability Test Sheet

Alexandra Holoborodko

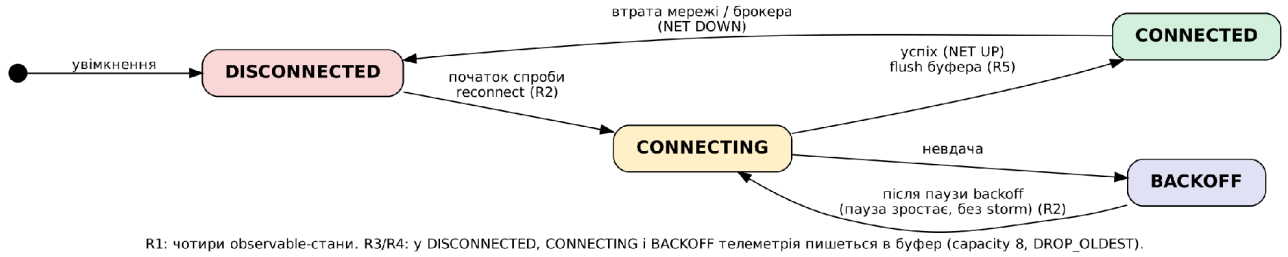
In short: the node restores the network, but violates 4 of 8 requirements (R2, R4, R6, R7). 4 defects were found, two of them Critical (silent data loss at the buffer limit and a memory leak of ≈80 bytes per message).

0. Environment

Parameter	Value
Tool	Wokwi Simulator in VS Code + PlatformIO (Serial Monitor, monitor_filters = time, log2file)
Firmware	homework_buggy (black box; conclusions drawn from logs and STATUS)
OS	Win 11
Log file	platformio-device-monitor-*.log (attached to the report); 8 runs in total, 13:19–14:16
Initial heap	350868 bytes

1. Network state model (R1)

Reference model per requirements R1, R2, R5 (initial state DISCONNECTED):



R1: чотири observable-стани. R3/R4: у DISCONNECTED, CONNECTING і BACKOFF телеметрія пишеться в буфер (capacity 8, DROP_OLDEST).

Actual transitions (from [NET] state=… in the log):

From state	Trigger	To state	What the log shows
CONNECTED	NET DOWN	DISCONNECTED	[NET] state=DISCONNECTED
DISCONNECTED	immediately (≤ 11 ms)	BACKOFF	state=BACKOFF; retry=1 backoff_ms=500
BACKOFF	timer (≈ 475 ms)	CONNECTING	state=CONNECTING
CONNECTING	failure	BACKOFF	reconnect=FAILED; state=BACKOFF; retry=N backoff_ms=50
BACKOFF / CONNECTING	NET UP	CONNECTED	[FAULT] network restored; state=CONNECTED; reconnect=SUCCESS (≈ 2 ms)

Comparison with expected: all four states are observable (R1 met). Deviations from the model: DISCONNECTED goes straight to BACKOFF, bypassing CONNECTING; after the first 500 ms pause the pause becomes 50 ms and never changes again (see BUG-01). During boot ([BOOT] IoT node ready) there are no [NET] state= lines; the first STATUS already shows CONNECTED.

2. Risks

ID	Risk	Requirement	Consequence	How I check	Result
RISK-01	The node does not recover after network loss	R2, R5	Telemetry does not arrive	Scenario A	Not confirmed (recovery in 2 ms)
RISK-02	Reconnect storm: attempts without backoff growth	R2	Load on the broker and Wi-Fi, battery drain	Scenario A, intervals measured from timestamps	Confirmed → BUG-01
RISK-03	Data loss while offline (buffer does not retain data)	R3, R5	Gaps in the measurement history	Scenario B	Not confirmed for ≤ 8 messages
RISK-04	Error at the buffer limit (>8, drop without a record)	R4	Silent data loss, corrupted flush	Scenario C (8, 9, 12)	Confirmed → BUG-03

RISK-05	Duplicate publish after reconnect	R6	Double records, wrong analytics	Counting publishes per ID	Confirmed → BUG-02
RISK-06	Memory leak under SEND/STRESS	R7	The node fails over time	Scenario D	Confirmed → BUG-04
RISK-07	STATUS does not reflect the real state	R8	Impossible to diagnose	STATUS in every scenario	Partially: the dropped field does not reflect losses

3. Scenario results

Scenario A — Connection loss / recovery (R2, R5)

Commands: RESET STATS → STATUS → NET DOWN → 10–15 s → STATUS → NET UP → STATUS. Executed 2 times.

Check	Expected	Actual	PASS/FAIL	Evidence
The node does not hang and accepts commands	Yes	STATUS in the BACKOFF state responded (net=BACKOFF, broker=false, reconnect_attempts=125)	PASS	13:20:01.607
Attempts in 10–15 s	A controlled number	204 attempts in ≈ 13.7 s (≈ 20 attempts/s)	FAIL	retry=1 13:19:51.818; retry=204 13:20:05.543
Intervals between attempts (from timestamps)	Growing / capped	First pause 500 ms, then a constant 50 ms: retry 2–125: 6.179 s / 123 = 50.2 ms; retry 126–204: 3.933 s / 78 = 50.4 ms	FAIL	BUG-01
After NET UP: state and NET LED	CONNECTED, LED on	CONNECTED after ≈ 2 ms, reconnect=SUCCESS, LED on (2/2)	PASS	13:20:08.538–.540
Heap	Stable	350868 before, during and after	PASS	STATUS

Scenario B — Offline buffer (R3, R5)

Commands: RESET STATS → NET DOWN → SEND ×N → BUFFER → NET UP → STATUS. Run 1: N=3 (two commands were mistyped, [CLI] ERROR unknown command), run 2: N=5.

Check	Expected	Actual (N=5, ID 4–8)	PASS/FAIL	Evidence
[BUFFER] queued for each SEND	5 entries	id=4…8, count=1/8…5/8	PASS	13:39:53–13:40:09
BUFFER	count=5	count=5 capacity=8 dropped=0	PASS	13:40:17
Flush after NET UP	All 5 IDs, buffer=0	flush_start count=5 → flush_end count=0, IDs 4–8 in order	PASS	13:40:24.719–.730
STATUS after	sent=5, duplicates=0	sent=10, duplicates=5	FAIL	BUG-02
Run 1 (N=3, IDs 1–3)	sent=3, duplicates=0	sent=6, duplicates=3	FAIL	13:36:30

Scenario C — Boundary buffer capacity (R4)

Commands: RESET STATS → NET DOWN → SEND ×N → BUFFER → NET UP → STATUS, for N = 8, 9, 12.

N	Check	Expected	Actual	PASS/FAIL
8 (ID 21–28)	count / dropped	8 / 0	count=8/8, dropped=0	PASS
8	Flush	IDs 21–28 once each	21–28 in order, but each ×2 (sent=16, duplicates=8)	FAIL (R6)
9 (ID 29–37)	count / dropped	8 / 1	count=9/8, dropped=0	FAIL
9	Flush	IDs 30–37 once each	37, 30, 31…36, 37 (29 missing, 37 twice), each ×2: sent=18, dropped=0, duplicates=9	FAIL
12 (ID 9–20)	count / dropped	8 / 4	count=12/8, dropped=0; log: overflow policy=DROP_OLDEST dropped_total=0	FAIL
12	Flush	IDs 13–20 once each	17–20 ×4, 13–16 ×2, IDs 9–12 missing; order 17,18,19,20,13,…,16,17,…,20	FAIL

Duplicate check (R6)

Run	Message ID	Publish count	Expected	OK / FAIL
B (5)	4, 5, 6, 7, 8	2 each	1	FAIL
C (8)	21–28	2 each	1	FAIL
C (9)	30–36	2 each	1	FAIL
C (9)	37	4	1	FAIL
C (9)	29	0 (lost)	1, or dropped with dropped=1	FAIL
C (12)	13–16	2 each	1	FAIL
C (12)	17–20	4 each	1	FAIL
C (12)	9–12	0 (lost)	dropped with dropped=4	FAIL
D (STRESS ×10)	1–1000	1 each	1	OK

Reconciliation: in every flush sent = 2 × the number of messages, duplicates = the number of messages. A duplicate appears only when sending from the buffer after reconnect; in the CONNECTED state (STRESS) there are no duplicates: 1000 messages, sent=1000, duplicates=0.

Scenario D — Network stress / Memory Stability Log (R7)

Commands: RESET STATS → STATUS → STRESS 100 + STATUS ×10. Network CONNECTED, buffer=0, dropped=0, duplicates=0 on all iterations.

Iter	Action	Network	Buffer	Sent	Dropped	Dup	Free heap	Δheap	Notes
0	Baseline	CONNECTED	0	0	0	0	350868	—	—
1	STRESS 100	CONNECTED	0	100	0	0	342604	-8264	82.6 B/msg
2	STRESS 100	CONNECTED	0	200	0	0	334604	-8000	80.0 B/msg
3	STRESS 100	CONNECTED	0	300	0	0	326612	-7992	79.9 B/msg
4	STRESS 100	CONNECTED	0	400	0	0	318600	-8012	80.1 B/msg
5	STRESS 100	CONNECTED	0	500	0	0	310600	-8000	80.0 B/msg
6	STRESS 100	CONNECTED	0	600	0	0	302600	-8000	80.0 B/msg
7	STRESS 100	CONNECTED	0	700	0	0	294600	-8000	80.0 B/msg
8	STRESS 100	CONNECTED	0	800	0	0	286600	-8000	80.0 B/msg
9	STRESS 100	CONNECTED	0	900	0	0	278600	-8000	80.0 B/msg
10	STRESS 100	CONNECTED	0	1000	0	0	270600	-8000	80.0 B/msg

Trend analysis: heap drops on every iteration and never recovers, −80268 bytes in total (≈ 23% of baseline) over 1000 messages. The rate is stable: ≈ 80 bytes per message (Δheap / 100), with no fluctuations in either direction. This is a systematic leak. At this rate memory runs out after roughly ≈ 4400 messages (extrapolation from the simulation). R7: FAIL (BUG-04).

4. Network State / Fault Log

Step	Command	State	Buffer	Sent	Dropped	Dup	Reconnects	Heap
1	RESET STATS	CONNECTED	0	0	0	0	0	350868
2	NET DOWN (+≈10 s), STATUS	BACKOFF	0	0	0	0	125	350868
3	NET UP, STATUS	CONNECTED	0	0	0	0	204	350868
4	B: SEND×3, NET UP, STATUS	CONNECTED	0	6	0	3	273	350364
5	B: SEND×5, NET UP, STATUS	CONNECTED	0	10	0	5	333	349964
6	C: SEND×12, STATUS (offline)	BACKOFF	12	0	0	0	546	349004
7	C: SEND×8, NET UP, STATUS	CONNECTED	0	16	0	8	369	348364
8	C: SEND×9, NET UP, STATUS	CONNECTED	0	18	0	9	323	347644

Note: STATUS after the flush in run C (12) was not captured, so it is not in the table.

5. Bug reports

BUG-01

Title	**Reconnect backoff does not grow: after NET DOWN the node retries every ~50 ms (reconnect storm)**
Requirement / Risk	R2 / RISK-02
Environment	Wokwi Simulator in VS Code + PlatformIO; firmware: homework_buggy; OS: Win 11
Preconditions	Node CONNECTED, STATS reset (RESET STATS), STATUS shows reconnect_attempts=0.
Steps	1. RESET STATS 2. NET DOWN 3. Wait 10–15 s and watch [NET] retry=… backoff_ms=… lines 4. STATUS 5. NET UP
Expected	Pause between reconnect attempts increases (e.g. 500 → 1000 → 2000 ms, capped) and does not create a reconnect storm (R2).
Actual	First pause is 500 ms (retry=1), from retry=2 on backoff_ms is always 50 ms. 204 attempts in ≈ 13.7 s (≈ 20 attempts per second). The log flood makes the serial console hard to use.
Recovery behavior	Node recovers by itself: after NET UP it reaches CONNECTED in ≈ 2 ms, reconnect=SUCCESS, NET LED on.
IDs / Counters / Timing	retry=1 at 13:19:51.818 (backoff_ms=500); retry=204 at 13:20:05.543 (backoff_ms=50). retry 2–125: 123 attempts in 6.179 s = 50.2 ms avg; retry 126–204: 78 attempts in 3.933 s = 50.4 ms avg. STATUS during outage: {"net":"BACKOFF","broker":false,…,"reconnect_attempts":125,"heap":350868}
Heap before/after	350868 / 350868 (no change)
Evidence	[NET] retry=1 backoff_ms=500 [NET] retry=2 backoff_ms=50 [NET] retry=3 backoff_ms=50 … [NET] retry=204 backoff_ms=50 Full log: platformio-device-monitor-*.log, 13:19:51–13:20:08
Severity	Major. On real hardware a 20 Hz reconnect loop loads the broker and Wi-Fi and drains the battery; there is no workaround on the device side.
Reproducibility	2/2 detailed runs with timestamps; the same 50 ms loop was seen in every later offline test (e.g. 639 attempts in ≈ 80 s).

BUG-02

Title	**Every buffered message is published twice after NET UP (duplicate publish during flush)**
Requirement / Risk	R6 / RISK-05
Environment	Wokwi Simulator in VS Code + PlatformIO; firmware: homework_buggy; OS: Win 11
Preconditions	Node CONNECTED, buffer=0, after RESET STATS.
Steps	1. RESET STATS 2. NET DOWN 3. SEND (5 times, one command each) 4. BUFFER (count=5) 5. NET UP 6. STATUS
Expected	Each message ID is successfully published once: sent=5, duplicates=0 (R6).
Actual	Every buffered ID appears twice in [MQTT] publish lines. STATUS: sent=10, duplicates=5. Not reproducible while CONNECTED (STRESS 100 ×10: sent=1000, duplicates=0), so the defect is in the flush after reconnect.
Recovery behavior	Node recovers by itself, buffer becomes 0, but duplicated data is sent.
IDs / Counters / Timing	IDs 4–8: each published 2×, 13:40:24.719–13:40:24.730. flush_start count=5 → flush_end count=0. Other runs: 3 messages → sent=6, duplicates=3; 8 messages → sent=16, duplicates=8; 9 messages → sent=18, duplicates=9.
Heap before/after	350364 → 349964 (after the 5-message flush; see BUG-04)
Evidence	[BUFFER] flush_start count=5 [MQTT] publish id=4 temp=24.5 [MQTT] publish id=4 temp=24.5 [MQTT] publish id=5 temp=24.5 [MQTT] publish id=5 temp=24.5 … {"net":"CONNECTED","broker":true,"buffer":0,"capacity":8,"sent":10,"dropped":0,"duplicates":5,"recon

	nect_attempts":333,"heap":349964}
Severity	Major. Telemetry is duplicated on the broker, which breaks counting and analytics; no workaround on the device.
Reproducibility	5/5 (every flush after reconnect in runs B ×2 and C ×3)

BUG-03

Title	**Buffer overflow: capacity exceeded, DROP_OLDEST not applied, oldest messages lost without dropped counter, flush sends a corrupted set**
Requirement / Risk	R4 (and R6, R8) / RISK-04
Environment	Wokwi Simulator in VS Code + PlatformIO; firmware: homework_buggy; OS: Win 11
Preconditions	Node CONNECTED, buffer=0, after RESET STATS; buffer capacity = 8.
Steps	1. RESET STATS 2. NET DOWN 3. SEND ×9 (also repeated with ×12) 4. BUFFER, STATUS 5. NET UP 6. STATUS
Expected	buffer never exceeds 8; with 9 messages dropped=1 (with 12: dropped=4) and a log entry; after NET UP the 8 newest IDs are published once each; no corrupted data (R4).
Actual	9 messages: queued id=37 count=9/8, BUFFER count=9 capacity=8 dropped=0; log says "overflow policy=DROP_OLDEST dropped_total=0" but nothing is dropped. Flush sends 37, 30, 31 … 36, 37 (ID 29 never published, ID 37 twice in the buffer), every publish duplicated; STATUS sent=18, dropped=0. 12 messages: count=12/8, dropped=0; flush publishes 17–20 (four times each), 13–16 (twice), IDs 9–12 never published. 8 messages: works (count=8/8, dropped=0).
Recovery behavior	Node reconnects by itself, but the lost messages are never reported.
IDs / Counters / Timing	N=9: IDs 29–37; 14:07:38 overflow log; 14:07:46 BUFFER count=9 dropped=0; flush 14:07:51.650–.672; STATUS sent=18 dropped=0 duplicates=9. N=12: IDs 9–20; 13:51:05.889 overflow log; 13:51:28 BUFFER count=12 dropped=0; STATUS (offline) buffer=12 capacity=8; flush 13:51:45.166–.221 (flush_start count=12).
Heap before/after	N=9: 348364 → 347644; N=12: 349004 before flush
Evidence	[BUFFER] queued id=36 count=8/8 [BUFFER] overflow policy=DROP_OLDEST dropped_total=0 [BUFFER] queued id=37 count=9/8 [BUFFER] count=9 capacity=8 dropped=0 [BUFFER] flush_start count=9 [MQTT] publish id=37 … id=30 … id=31 … id=36 … id=37 {"buffer":0,"capacity":8,"sent":18,"dropped":0,"duplicates":9,…}
Severity	Critical. Data is lost silently (the dropped counter stays 0), the buffer grows beyond its capacity, and the flushed data is wrong.
Reproducibility	2/2 (N=9 and N=12); boundary N=8 passes

BUG-04

Title	**Memory leak: free heap drops by ≈ 80 bytes per message and never recovers**
Requirement / Risk	R7 / RISK-06
Environment	Wokwi Simulator in VS Code + PlatformIO; firmware: homework_buggy; OS: Win 11
Preconditions	Simulator freshly restarted; node CONNECTED; RESET STATS; STATUS gives baseline heap=350868.
Steps	1. RESET STATS, STATUS (iteration 0) 2. STRESS 100, STATUS 3. Repeat step 2 ten times in total
Expected	No systematic irreversible drop of free heap on repeated SEND/STRESS; small fluctuations in both directions are acceptable (R7).
Actual	Heap decreases on every iteration by ≈ 8000 bytes (≈ 80 bytes per message) and never goes back: 350868 → 270600 after 1000 messages (−80268 bytes, ≈ 23%). At this rate memory would be exhausted after ≈ 4400 messages (extrapolation).

Recovery behavior	Heap is not released after the stress finishes; it returns to the baseline only after a simulator restart.
IDs / Counters / Timing	Δheap per iteration: −8264, −8000, −7992, −8012, −8000, −8000, −8000, −8000, −8000, −8000. Each iteration sent += 100, dropped=0, duplicates=0, reconnect_attempts=0.
Heap before/after	350868 → 270600
Evidence	[STRESS] start count=100 heap=350868 [STRESS] end heap=342604 [STRESS] start count=100 heap=342604 [STRESS] end heap=334604 … [STRESS] end heap=270600 Log 14:13:43–14:16:20
Severity	Critical. The node stops working after enough messages (memory exhaustion).
Reproducibility	10/10 iterations in one run; heap also fell after every flush in scenarios B and C (350868 → 347644).

6. Conclusion

Is the IoT node stable in a non-ideal network? No. The node survives network loss and recovery: it does not hang, accepts commands and returns to CONNECTED by itself within a few milliseconds. However, it violates four of eight requirements: it floods the network with attempts (R2), misbehaves at the buffer limit (R4), duplicates messages after reconnect (R6) and leaks memory (R7).

Requirement	Result	Based on test
R1 Network states	PASS	Scenario A: all 4 states are observable in [NET] state=…
R2 Auto-reconnect / backoff	FAIL	Scenario A: backoff 500 → 50 ms, ≈ 20 attempts/s (BUG-01). The node does not hang
R3 Offline buffering	PASS	Scenario B, C(8): queued is counted correctly, flush delivers all IDs (for ≤ 8)
R4 Buffer boundary (DROP_OLDEST)	FAIL	Scenario C(9, 12): count>8, dropped=0, IDs are lost (BUG-03)
R5 Recovery	PASS	Scenario A, B: CONNECTED in ≈ 2 ms, flush is performed; for flush quality see R4 and R6
R6 Duplicates	FAIL	Scenario B, C: each ID ×2, sent=2×N (BUG-02)
R7 Memory stability	FAIL	Scenario D: −8000 bytes per 100 messages, no recovery (BUG-04)
R8 Evidence (STATUS)	PASS	All fields are present; reconnect_attempts matches retry in the log. Caveat: dropped does not reflect real losses (BUG-03)

The most critical risks for a real product:

1) Silent data loss at the buffer limit (BUG-03): in a normal network outage part of the telemetry disappears and the dropped counter does not show it, so the problem cannot be noticed. 2) Memory leak (BUG-04): after a few thousand messages the device will fail, and with the network down (when more messages sit in the buffer) this will happen sooner. 3) Reconnect storm (BUG-01) and duplicates (BUG-02): load on the broker and battery, and double data.

What should be verified on real hardware:

Real Wi-Fi and an MQTT broker, where a failed reconnect takes seconds (not ≈ 45 ms as in the simulator), and therefore the real attempt rate; long runs (soak tests of hours and days) with heap and watchdog monitoring;

reboot and power loss with a non-empty buffer (is the data kept, given that the buffer is in RAM); different QoS levels and broker behaviour on duplicates; real delays and drops during flush.

7. Appendices

• Serial Monitor log file: platformio-device-monitor-*.log (attached to the report).